

Report on Corpus Analysis Program

OVERVIEW

- * I have chosen to build a Corpus Frequency Word Analyser.
- * Given that topics like regex have not been discussed yet, I have stuck to using a few basic concepts in order to build the Corpus Frequency Word Analyser.
- * This approach will also illustrate the significance of Regular Expressions and other dedicated libraries in NLP.
- * The program utilises the following concepts mainly : while loop , for loop , if loop , file i/o
- * Apart from this, the `string` library has been imported for convenience.
- * At the end of the program, an if loop allowing for automatic processing and modular use has been included.

METADATA OF THE CORPUS USED

- * The corpus used in the project is a text file encoded in ASCII. The corpus is in the English Language.
- * Content-wise, it is a short story pertaining to Sherlock Holmes, titled The Adventure of the Veiled Dodger, with the author being Arthur Conan Doyle.
- * The entire corpus is about 26 kilobytes, with a line count of 518, including empty lines, titles, and disclaimers.
- * The corpus can be accessed online via the following url :
 - <https://sherlock-holm.es/stories/plain-text/veil.txt>
- * Given that there is a disclaimer at the end of the original corpus, I have used an edited corpus without the disclaimer.
- * A diff between the original and the edited one has been included in the next slide

```
viswas@admin ~/Projects/Python/NLP-EFL › diff corpus.txt veil.txt
501a502,512
> -----
> This text is provided to you "as-is" without any warranty. No
> warranties of any kind, expressed or implied, are made to you as to
> the text or any medium it may be on, including but not limited to
> warranties of merchantability or fitness for a particular purpose.
>
> This text was formatted from various free ASCII and HTML variants.
> See http://sherlock-holm.es for an electronic form of this text and
> additional information about it.
>
> This text comes from the collection's version 3.1.
```

Image 1: 'diff' is a GNU utility used to find differences between files. Here corpus.txt is the edited corpus and veil.txt is the original one, with the disclaimer. The output of this command shows the difference between both the files along with the line number from where the difference between both originate.

```

vanilla.py ✎
1 import string
2
3 def basic_analysis(filename):
4     with open(filename, 'r', encoding='utf-8') as f:
5         text = f.read()
6
7         text = text.lower()
8
9         words = text.split()
10        words_clean = []
11        for w in words:
12            w = w.strip(string.punctuation)
13            if w:
14                words_clean.append(w)
15
16        word_counts = {}
17        for w in words_clean:
18            word_counts[w] = word_counts.get(w, 0) + 1
19
20        sentence_endings = 0
21        for char in text:
22            if char in '.!?:':
23                sentence_endings += 1
24        sentence_count = sentence_endings if sentence_endings > 0 else 1
25
26        total_chars = sum(len(w) for w in words_clean)
27        avg_word_len = total_chars / len(words_clean) if words_clean else 0
28
29        sorted_words = sorted(word_counts.items(), key=lambda x: x[1], reverse=True)
30
31        print("Top 20 words:")
32        for word, freq in sorted_words[:20]:
33            print(f"{word}: {freq}")
34
35        with open('analysis_basic.txt', 'w', encoding='utf-8') as out:
36            out.write(f"Sentence count: {sentence_count}\n")
37            out.write(f"Average word length: {avg_word_len:.2f}\n")
38            out.write("Top 20 words:\n")
39            for word, freq in sorted_words[:20]:
40                out.write(f"{word}: {freq}\n")
41
42    if __name__ == "__main__":
43        basic_analysis("corpus.txt")
44
45

```

Image 2: This is the entire code snippet. The following 4 slides will discuss how the program does what it does.

METHODOLOGY (1/4)

- * The program defines a function wherein the file is opened in read-only mode.
- * It uses the utf-8 encoding, since the text is in plaintext.
- * Preprocessing involves simple string based methods.
- * It uses the `.lower()` method to standardize the corpus, by turning everything lowercase.
- * Basic tokenization is achieved via the `.split()` method.
- * However, this will later on pose issues, since this does not split hyphenated words and hence words like well-known would be counted as one single token instead of two.
- * While the concept has not been introduced yet in Python, regular expressions are better for tokenization, albeit better alternatives are provided via libraries like nltk.

METHODOLOGY (2/4)

- * Once tokenisation is done, the corpus has to be sanitized.
- * All tokenized words are stored in a list, and from the list, all punctuations are stripped via a for loop, and the cleaned data is put into a new list.
- * Then, a dictionary is made, wherein words from the cleaned data set are added. If the entry is new, the value defaults to 1, and then is incremented for every other instance of the word. This is how the word frequency counter is implemented.
- * Sentence count is implemented via a for loop and an if loop. For every occurrence of a full stop(.), exclamation mark (!), and question mark (?), the `sentence_ending` count is incremented. If, at the end of going through the unprocessed data, there is no occurrence of any of these punctuations, the sentence counter defaults to 1.

METHODOLOGY (3/4)

- * Calculating total characters includes simply getting the sum of the length of every word from the `words_clean` list, i.e the list that is stripped of punctuations, empty spaces, and new lines.
- * Average word length is obtained via dividing the total number of characters with the length of the `word_clean` list.
- * Now, there's a need for getting a ranked dictionary so as to produce the top 20 occurring words.
- * This is done via a lambda function here, wherein the sorting key is the value of each key-value pair. Else, the sorting would be done alphabetically. The reverse flag here allows for a descending list, which is exactly what we need.
- * The top 20 words are then printed by getting entries 0 to 19 of the sorted dictionary.

METHODOLOGY (4/4)

- * The required metrics are then written to a file, while the frequency is printed before this process.
- * Here, the program utilises an already defined file name, however, variables can be set via the `input()` function, or using the sys library, if required.
- * Finally, towards the end, a stub has been added that automatically reads corpus.txt if the program is called directly. However, if needed, the entire program can be called as a library, and then `basic_analysis()` becomes a function that can be called upon in other programs. However, of this is the case, the file name has to be provided and corpus.txt won't be automatically read.

IMPLEMENTATION ISSUES

- * Mostly, there were none, however, laziness being a big issue, I chose to import the string library to list out all punctuation for data sanitation purposes instead of simply listing them all out.
- * Another thing to note is that this program while functional is not efficient enough for production use, since I have chosen not to include libraries not yet taught.
- * Production ready code would utilise nltk or re (regex library).

OUTPUT

```
viswas@admin ~/Projects/Python/NLP-EFL > python vanilla.py  
Top 20 words:  
the: 248  
and: 139  
i: 127  
of: 115  
to: 109  
was: 99  
it: 93  
a: 91  
that: 87  
in: 62  
he: 59  
is: 53  
you: 46  
my: 41  
had: 41  
have: 36  
her: 36  
but: 35  
his: 33  
she: 33
```

```
viswas@admin ~/Projects/Python/NLP-EFL > cat analysis_basic.txt  
Sentence count: 358  
Average word length: 4.10  
Top 20 words:  
the: 248  
and: 139  
i: 127  
of: 115  
to: 109  
was: 99  
it: 93  
a: 91  
that: 87  
in: 62  
he: 59  
is: 53  
you: 46  
my: 41  
had: 41  
have: 36  
her: 36  
but: 35  
his: 33  
she: 33
```